

Optimización Estocástica

Yael Rene Santiago Cruz

1. Algoritmo EM

En un contexto frecuentista, consideramos un modelo probabilístico con densidad f_θ . El estimador de máxima verosimilitud se obtiene maximizando la función de verosimilitud

$$L(\theta; x_1, \dots, x_n) = \prod_{i=1}^n f_\theta(x_i),$$

lo cual puede resultar complicado en presencia de variables latentes.

Para abordar este problema, se introduce una variable latente X^f (o Z en muchos contextos) y se define la verosimilitud completa

$$L_c(\theta; x, x^f) = \prod_{i=1}^n f_c(x_i, x_i^f),$$

la cual suele ser más sencilla de maximizar.

El algoritmo EM (Esperanza-Maximización) es un procedimiento iterativo que permite aproximar el estimador de máxima verosimilitud a partir de esta formulación.

1.1. Algoritmo EM

1. Inicializar $\theta^{(0)}$.

2. Para $k \geq 1$, iterar:

▪ **Paso E (Esperanza):**

$$Q(\theta \mid \theta^{(k-1)}) = \mathbb{E}_{\theta^{(k-1)}} [\log L_c(\theta; X, X^f) \mid X = x].$$

▪ **Paso M (Maximización):**

$$\theta^{(k)} = \arg \max_{\theta \in \Theta} Q(\theta \mid \theta^{(k-1)}).$$

3. Repetir hasta convergencia.

El algoritmo EM satisface la propiedad de monotonía:

$$L(\theta^{(k)}; x) \geq L(\theta^{(k-1)}; x).$$

1.1.1. Ejemplo: Mezcla de Gaussianas

Sea el modelo:

$$Z_i \sim \text{Categorical}(p_1, \dots, p_d), \quad X_i \mid Z_i = j \sim \mathcal{N}(\mu_j, \sigma_j^2).$$

La verosimilitud completa es:

$$\log L_c(\mu, p, \sigma^2; x, z) = \sum_{i=1}^n \sum_{j=1}^d \mathbf{1}_{\{z_i=j\}} (\log p_j + \log \mathcal{N}(x_i \mid \mu_j, \sigma_j^2)).$$

Paso E

El paso E consiste en calcular:

$$Q(\theta \mid \theta^{(k-1)}) = \mathbb{E}_{\theta^{(k-1)}} [\log L_c(\theta; X, Z) \mid X = x].$$

Sustituyendo la expresión de la verosimilitud completa:

$$Q(\theta \mid \theta^{(k-1)}) = \sum_{i=1}^n \sum_{j=1}^d \mathbb{E}_{\theta^{(k-1)}} [\mathbf{1}_{\{Z_i=j\}} \mid X_i = x_i] (\log p_j + \log \mathcal{N}(x_i \mid \mu_j, \sigma_j^2)).$$

Definiendo

$$\gamma_{ij}^{(k-1)} = \mathbb{P}_{\theta^{(k-1)}}(Z_i = j \mid X_i = x_i),$$

obtenemos

$$Q(\theta \mid \theta^{(k-1)}) = \sum_{i=1}^n \sum_{j=1}^d \gamma_{ij}^{(k-1)} (\log p_j + \log \mathcal{N}(x_i \mid \mu_j, \sigma_j^2)).$$

Las cantidades γ_{ij} se conocen como *responsabilidades* y se calculan mediante

$$\begin{aligned} \gamma_{ij} &= \mathbb{P}_{\theta^{(k-1)}}(Z_i = j \mid X_i = x_i) \\ &= \frac{\mathbb{P}_{\theta^{(k-1)}}(X_i = x_i \mid Z_i = j) \mathbb{P}_{\theta^{(k-1)}}(Z_i = j)}{\mathbb{P}_{\theta^{(k-1)}}(X_i = x_i)} \\ &= \frac{p_j \mathcal{N}(x_i \mid \mu_j, \sigma_j^2)}{\sum_{m=1}^d p_m \mathcal{N}(x_i \mid \mu_m, \sigma_m^2)} \end{aligned}$$

Paso M

Definiendo

$$n_j = \sum_{i=1}^n \gamma_{ij},$$

las actualizaciones que maximizan Q son:

$$p_j^{(k)} = \frac{n_j}{n},$$

$$\mu_j^{(k)} = \frac{1}{n_j} \sum_{i=1}^n \gamma_{ij} x_i,$$

$$(\sigma_j^2)^{(k)} = \frac{1}{n_j} \sum_{i=1}^n \gamma_{ij} (x_i - \mu_j^{(k)})^2.$$

El algoritmo EM puede interpretarse como un método de optimización que alterna entre el cálculo de esperanzas (Paso E) y la maximización de una función auxiliar (Paso M). En casos donde el Paso M no puede resolverse analíticamente, es posible realizar una actualización mediante ascenso por gradiente:

$$\theta^{(k)} = \theta^{(k-1)} + \gamma^{(k)} \nabla_{\theta} Q(\theta \mid \theta^{(k-1)}),$$

lo cual conecta el algoritmo EM con métodos de optimización estocástica.

Estas expresiones se generalizan de manera natural al caso multivariado, donde $x_i \in \mathbb{R}^d$. En este contexto:

- x_i es un vector en $\mathbb{R}^d$,
- $\mu_j \in \mathbb{R}^d$ es el vector de medias,
- $\Sigma_j \in \mathbb{R}^{d \times d}$ es la matriz de covarianza.

Las responsabilidades toman la forma

$$\gamma_{ij} = \frac{p_j \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}{\sum_{m=1}^K p_m \mathcal{N}(x_i \mid \mu_m, \Sigma_m)}.$$

Las actualizaciones en el paso M son:

$$p_j^{(k)} = \frac{n_j}{n}, \quad \mu_j^{(k)} = \frac{1}{n_j} \sum_{i=1}^n \gamma_{ij} x_i,$$

$$\Sigma_j^{(k)} = \frac{1}{n_j} \sum_{i=1}^n \gamma_{ij} (x_i - \mu_j^{(k)})(x_i - \mu_j^{(k)})^T.$$

2. Descenso de Gradiente Estocástico (SGD)

Un método para encontrar los estimadores máximo verosímiles es el *descenso del gradiente estocástico* (SGD por sus siglas en inglés). El descenso de gradiente estocástico y sus variantes han demostrado éxito en muchas aplicaciones de aprendizaje automático. Este éxito motiva el análisis de sus propiedades estadísticas y de convergencia.

Consideramos el problema estándar de optimización:

$$\min_{\theta \in \mathbb{R}^d} \mathbb{E}_{\xi \sim P}[\ell(\theta, \xi)],$$

donde $\xi : \Omega \rightarrow E$ es una variable aleatoria y $\ell : \mathbb{R}^d \times E \rightarrow \mathbb{R}$ es una función de pérdida fuertemente convexa. Sea θ^* el minimizador global de este problema.

La actualización de SGD es:

$$\theta_{n+1} = \theta_n - \eta_n \nabla \ell(\theta_n, \xi_{n+1}),$$

con $\{\xi_n\}_{n \geq 1}$ i.i.d. y

$$\mathbb{E}[\nabla \ell(\theta_n, \xi_{n+1}) \mid \sigma(\xi_1, \dots, \xi_n)] = \mathbb{E}_{\xi \sim P}[\nabla \ell(\theta_n, \xi)].$$

Convergencia casi segura

Bajo condiciones apropiadas (como tasas de aprendizaje decrecientes que satisfacen $\sum \eta_n = \infty$ y $\sum \eta_n^2 < \infty$, y convexidad fuerte de ℓ), se tiene que $\theta_n \rightarrow \theta^*$ casi seguramente, donde θ^* es el mínimo global.

Distribución asintótica

Cuando $\nabla \ell(\theta, \xi)$ tiene varianza finita, la distribución asintótica satisface

$$\eta_n^{-1/2}(\theta_n - \theta^*) \Rightarrow \mathcal{N}(0, \Sigma),$$

donde Σ satisface la ecuación de Lyapunov:

$$\mathbb{E}[\nabla^2 \ell(\theta^*, \xi)]\Sigma + \Sigma \mathbb{E}[\nabla^2 \ell(\theta^*, \xi)] = \Gamma,$$

con $\Gamma = \text{Cov}(\nabla \ell(\theta^*, \xi), \nabla \ell(\theta^*, \xi))$. Equivalentemente, esta es la distribución estacionaria del proceso de Ornstein–Uhlenbeck:

$$dZ_t = -\mathbb{E}[\nabla^2 \ell(\theta^*, \xi)]Z_t dt + \Gamma^{1/2} dB_t.$$

Versión Mini-batch

En la práctica, se usa frecuentemente una variante llamada *SGD con mini-batch*. En lugar de usar una sola observación ξ_{n+1} para estimar el gradiente, se utiliza un pequeño lote (batch) de m observaciones independientes $\{\xi_{n+1}^{(1)}, \dots, \xi_{n+1}^{(m)}\}$, y la actualización es:

$$\theta_{n+1} = \theta_n - \frac{\eta_n}{m} \sum_{j=1}^m \nabla \ell(\theta_n, \xi_{n+1}^{(j)}).$$

2.1. SGD para la optimización de funciones objetivo de sumas finitas

Supongamos que tenemos un conjunto de entrenamiento

$$\mathcal{D} = \{(x_1, y_1), \dots, (x_N, y_N) : x_i \in \mathcal{X}, y_i \in \mathcal{Y}\}$$

y consideremos el problema de *minimización del riesgo empírico*, el cual requiere minimizar

$$\mathcal{L}(\theta) = \mathbb{E}_{P(x,y)}[l(y, f(x, \theta))] \approx \frac{1}{N} \sum_{i=1}^N l(y_i, f(x_i, \theta)).$$

Dado que la esperanza no depende de los parámetros, podemos meter el gradiente dentro de la integral y aproximarlos mediante Monte Carlo usando un *minibatch* de tamaño $B = |\mathcal{B}|$, con $\mathcal{B} \in \mathcal{D}$, es decir

$$\nabla_{\theta} \mathcal{L}(\theta) = \nabla_{\theta} \mathbb{E}_{P(x,y)}[l(y, f(x, \theta))] = \mathbb{E}_{P(x,y)}[\nabla_{\theta} l(y, f(x, \theta))] \approx \frac{1}{B} \sum_{n \in \mathcal{B}} \nabla_{\theta} l(y, f(x, \theta)).$$

2.2. Gradientes en modelos de la familia exponencial de dispersión

Supongamos que la variable respuesta y sigue una distribución de la familia exponencial de dispersión (EDM), cuya función de probabilidad puede escribirse como

$$p(y; \theta, \phi) = a(y, \phi) \exp \left(\frac{y\theta - \kappa(\theta)}{\phi} \right),$$

donde θ es el parámetro canónico, $\kappa(\theta)$ es la función cumulante y ϕ es el parámetro de dispersión.

Definimos la función de pérdida como el negativo del logaritmo de la verosimilitud. Ignorando términos constantes que no dependen de los parámetros, la pérdida para una observación se reduce a

$$\ell(y, \theta) = \kappa(\theta) - y\theta.$$

Derivando con respecto a θ , se obtiene

$$\nabla_{\theta} \ell(y, \theta) = \kappa'(\theta) - y.$$

Una propiedad fundamental de las EDM es que

$$\kappa'(\theta) = \mu = \mathbb{E}[y],$$

por lo que el gradiente toma la forma

$$\boxed{\nabla_{\theta} \ell(y, \theta) = \mu - y.}$$

Aplicando la regla de la cadena, el gradiente respecto a la media es

$$\nabla_{\mu}\ell(y, \mu) = \nabla_{\theta}\ell(y, \theta) \cdot \frac{d\theta}{d\mu}.$$

Dado que

$$\frac{d\mu}{d\theta} = \kappa''(\theta),$$

se tiene

$$\boxed{\nabla_{\mu}\ell(y, \mu) = \frac{\mu - y}{\kappa''(\theta)}.$$

Consideremos ahora un modelo supervisado donde

$$\eta = x^{\top}\beta, \quad \mu = g^{-1}(\eta),$$

y g es la función de enlace. En la familia exponencial se cumple que

$$\mu = \kappa'(\theta),$$

por lo que el link canónico viene dado por

$$\boxed{g(\mu) = \theta = (\kappa')^{-1}(\mu)}.$$

El gradiente respecto a los parámetros del modelo se obtiene mediante la regla de la cadena:

$$\nabla_{\beta}\ell(y, f(x, \beta)) = \frac{\partial\ell}{\partial\theta} \cdot \frac{\partial\theta}{\partial\eta} \cdot \frac{\partial\eta}{\partial\beta} = (\mu - y) \frac{\partial\theta}{\partial\eta} x.$$

En el caso del link canónico se cumple que $\theta = \eta$, y por tanto

$$\frac{\partial\theta}{\partial\eta} = 1,$$

lo que implica

$$\boxed{\nabla_{\beta}\ell(y, f(x, \beta)) = (\mu - y)x.}$$

Si el link no es canónico, entonces $\theta \neq \eta$ y aparece un factor adicional:

$$\frac{\partial\theta}{\partial\eta} = \frac{d\theta}{d\mu} \cdot \frac{d\mu}{d\eta} = \frac{1}{\kappa''(\theta)} \cdot \frac{d\mu}{d\eta}.$$

Por lo tanto, el gradiente general es

$$\boxed{\nabla_{\beta}\ell = (\mu - y) \frac{1}{\kappa''(\theta)} \frac{d\mu}{d\eta} x.}$$

A continuación, se muestra las algunas densidades que pertenecen a esta familia:

- **Normal** $\mathcal{N}(\mu, \sigma^2)$ (con σ^2 conocida):

$$p(y; \theta, \phi) = \frac{1}{\sqrt{2\pi\phi}} \exp \left(\frac{y\theta - \frac{\theta^2}{2}}{\phi} - \frac{y^2}{2\phi} \right)$$

$$\theta = \mu, \quad \phi = \sigma^2$$

$$\kappa(\theta) = \frac{\theta^2}{2}, \quad \kappa'(\theta) = \theta = \mu$$

$$a(y, \phi) = \frac{1}{\sqrt{2\pi\phi}} \exp \left(-\frac{y^2}{2\phi} \right)$$

$$\ell(y, \mu) = \frac{(y - \mu)^2}{2}$$

- **Bernoulli** (p):

$$p(y; \theta) = \exp(y\theta - \log(1 + e^\theta)), \quad y \in \{0, 1\}$$

$$\theta = \log \left(\frac{p}{1-p} \right), \quad \phi = 1$$

$$\kappa(\theta) = \log(1 + e^\theta), \quad \kappa'(\theta) = \frac{e^\theta}{1 + e^\theta} = \mu$$

$$a(y, \phi) = 1$$

$$\ell(y, \mu) = -y \log \mu - (1 - y) \log(1 - \mu)$$

- **Poisson** (λ):

$$p(y; \theta) = \frac{1}{y!} \exp(y\theta - e^\theta), \quad y \in \mathbb{N}$$

$$\theta = \log \lambda, \quad \phi = 1$$

$$\kappa(\theta) = e^\theta, \quad \kappa'(\theta) = e^\theta = \mu$$

$$a(y, \phi) = \frac{1}{y!}$$

$$\ell(y, \mu) = \mu - y \log \mu$$

- **Exponencial** (λ):

$$p(y; \theta) = \exp(y\theta - \log(-\theta)), \quad y > 0$$

$$\theta = -\lambda, \quad \phi = 1$$

$$\kappa(\theta) = -\log(-\theta), \quad \kappa'(\theta) = -\frac{1}{\theta} = \mu$$

$$a(y, \phi) = 1$$

$$\ell(y, \mu) = \frac{y}{\mu} + \log \mu$$

- **Gamma** (parametrización tipo GLM):

$$p(y; \theta, \phi) = \exp\left(\frac{y\theta - (-\log(-\theta))}{\phi}\right) \cdot \frac{y^{\frac{1}{\phi}-1}}{\Gamma\left(\frac{1}{\phi}\right)}$$

$$\theta = -\frac{1}{\mu}, \quad \phi > 0$$

$$\kappa(\theta) = -\log(-\theta), \quad \kappa'(\theta) = -\frac{1}{\theta} = \mu$$

$$a(y, \phi) = \frac{y^{\frac{1}{\phi}-1}}{\Gamma\left(\frac{1}{\phi}\right)}$$

$$\ell(y, \mu) = \frac{y}{\mu} + \log \mu$$

3. Búsqueda Aleatoria

Una estrategia sorprendentemente efectiva en problemas donde no sabemos nada acerca del objetivo es usar búsqueda aleatoria. En este enfoque, cada iteración

$$x_{t+1}$$

se elige de manera uniforme al azar de

$$\mathcal{X}.$$

Esto siempre debería intentarse como línea base.

Se ha aplicado esta técnica al problema de la optimización de hiperparámetros para algunos modelos de aprendizaje automático, donde el objetivo es el rendimiento en un conjunto de validación. En estos ejemplos, el espacio de búsqueda es continuo,

$$\Theta = [0, 1]^D,$$

lo que hace sencillo muestrear de manera aleatoria. La alternativa estándar consiste en cuantizar el espacio en un conjunto fijo de valores y luego evaluarlos todos; esto se conoce como *grid search*. Por supuesto, esto solo es factible si el número de dimensiones D es pequeño.

Se encontró que la búsqueda aleatoria superaba al *grid search*. La razón intuitiva de esto es que muchos hiperparámetros no hacen mucha diferencia en la función objetivo, y en consecuencia resulta una pérdida de tiempo colocar una cuadrícula fina a lo largo de dimensiones poco importantes.

4. Recocido Simulado

El recocido simulado es un algoritmo estocástico de búsqueda local que intenta encontrar el mínimo global de una función de caja negra $E(x)$, conocida como función de energía. El método convierte la energía en una distribución de probabilidad no normalizada sobre los estados, definida como

$$p(x) = \exp(-E(x)),$$

y utiliza una variante del algoritmo de Metropolis-Hastings para muestrear de un conjunto de distribuciones de probabilidad. Al final, el método tiende a muestrear de los modos de la distribución, es decir, de los estados más probables o de menor energía. Este enfoque puede aplicarse tanto a optimización discreta como continua.

La idea se inspira en el proceso físico de recocido: calentar un sólido y enfriarlo lentamente hasta alcanzar un estado de mínima energía. Para trasladar este concepto a la optimización, se define

$$p_T(x) = \exp\left(-\frac{E(x)}{T}\right),$$

donde T es la temperatura, que se reduce gradualmente. A medida que T disminuye, los picos más grandes se acentúan y los más pequeños desaparecen, permitiendo seguir el máximo dominante y encontrar el óptimo global.

En cada paso, se propone un nuevo estado

$$x' \sim q(\cdot|x_t),$$

que para parámetros reales suele ser un paseo aleatorio

$$x' = x_t + \epsilon_{t+1}, \quad \epsilon_{t+1} \sim \mathcal{N}(0, \Sigma).$$

La probabilidad de aceptación se calcula como

$$\alpha_{t+1} = \exp\left(-\frac{E(x') - E(x_t)}{T_t}\right),$$

y se acepta el nuevo estado con probabilidad $\min(1, \alpha_{t+1})$. Si el nuevo estado tiene menor energía, se acepta siempre; si tiene mayor energía, puede aceptarse dependiendo de la temperatura.

La velocidad de enfriamiento se denomina *programa de enfriamiento*. Se ha demostrado que un enfriamiento logarítmico,

$$T_t \propto \frac{1}{\log(t+1)},$$

garantiza encontrar el óptimo global bajo ciertas condiciones, aunque es demasiado lento en la práctica. Por ello, se suele usar un enfriamiento exponencial

$$T_{t+1} = \gamma T_t, \quad \gamma \in (0, 1],$$

donde enfriar demasiado rápido puede llevar a quedar atrapado en un óptimo local, mientras que enfriar demasiado lento desperdicia tiempo. Determinar el mejor programa de enfriamiento es uno de los principales desafíos del recocido simulado.

Algorithm 1 Algoritmo de Recocido Simulado

- 1: Definir un estado inicial x_0 y temperatura inicial T_0
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Proponer un nuevo estado $x' \sim q(\cdot|x_{t-1})$
- 4: Calcular la probabilidad de aceptación

$$\alpha = \exp\left(-\frac{E(x') - E(x_{t-1})}{T_{t-1}}\right)$$

- 5: Aceptar $x_t = x'$ con probabilidad $\min(1, \alpha)$, en otro caso $x_t = x_{t-1}$
- 6: Actualizar la temperatura según el programa de enfriamiento, por ejemplo:

$$T_t = \gamma T_{t-1}, \quad \gamma \in (0, 1]$$

7: **end for**

4.1. Aplicación: búsqueda de hiperparámetros en redes neuronales

Una aplicación importante del recocido simulado es la optimización de hiperparámetros en modelos de aprendizaje automático, donde la función objetivo suele ser no convexa, costosa de evaluar y sin gradiente disponible respecto a varios parámetros estructurales.

Sea x el estado del algoritmo, que representa una configuración completa de hiperparámetros de una red neuronal. En este caso se considera

$$x = (\eta, \lambda, o, a, h),$$

donde:

- η es la tasa de aprendizaje,
- λ es el parámetro de regularización L_2 ,
- o es el optimizador (por ejemplo SGD o Adam),

- a es la función de activación,
- $h = (h_1, \dots, h_L)$ describe la arquitectura oculta, donde L es el número de capas y h_i el número de neuronas de cada capa.

El objetivo es minimizar una función de energía definida como la pérdida de validación del modelo entrenado con dichos hiperparámetros. Si f_x representa la red entrenada bajo la configuración x , se define

$$E(x) = \mathcal{L}_{\text{valid}}(f_x),$$

donde $\mathcal{L}_{\text{valid}}$ es la pérdida evaluada sobre el conjunto de validación.

Dado el estado actual x_t , se genera una propuesta x' mediante un kernel de transición $q(\cdot|x_t)$ que modifica aleatoriamente uno de los hiperparámetros. En este caso el kernel mezcla perturbaciones continuas y cambios discretos.

El kernel de transición se construye como una mezcla de kernels, seleccionando primero de manera uniforme cuál hiperparámetro modificar. Si $\mathcal{M}$ es el conjunto de movimientos posibles, entonces

$$J_t \sim \text{Uniform}(\mathcal{M}),$$

y, condicionado en $J_t = j$, se aplica el kernel correspondiente $q_j(\cdot|x_t)$. Así, el kernel total se escribe como

$$q(x'|x_t) = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} q_j(x'|x_t).$$

En este caso, los movimientos corresponden a modificar la tasa de aprendizaje, regularización, optimizador, función de activación, número de capas o número de neuronas. Una vez seleccionada una componente, sólo esa coordenada se actualiza, manteniendo fijas las demás.

Para la tasa de aprendizaje se utiliza una perturbación en escala logit:

$$z_t = \log \left(\frac{\eta_t}{1 - \eta_t} \right),$$

y se propone

$$z' = z_t + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2),$$

obteniendo

$$\eta' = \frac{1}{1 + e^{-z'}}.$$

De manera similar, para el parámetro de regularización se emplea una perturbación en escala logarítmica:

$$u_t = \log(\lambda_t),$$

$$u' = u_t + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2),$$

$$\lambda' = e^{u'}.$$

Para variables discretas, el kernel realiza saltos aleatorios:

- cambio de optimizador:

$$o' \in \{\text{SGD}, \text{Adam}\},$$

alternando entre ambos;

- cambio de activación:

$$a' \in \{\text{ReLU}, \text{Tanh}, \text{GELU}, \text{ELU}\},$$

seleccionando una distinta a la actual;

- modificación estructural:

se agrega o elimina una capa oculta,

$$L' = L \pm 1,$$

restringiendo el número total de capas;

- ajuste de ancho: se selecciona una capa j y se modifica su tamaño mediante

$$h'_j = h_j \cdot c,$$

donde $c \in \{0.5, 2\}$.

Así, el kernel de transición combina propuestas locales en un espacio híbrido continuo-discreto, permitiendo explorar arquitecturas y parámetros de entrenamiento.

Una vez propuesta la nueva configuración x' , se entrena la red y se calcula su energía $E(x')$. La probabilidad de aceptación es

$$\alpha_t = \min \left\{ 1, \exp \left(-\frac{E(x') - E(x_t)}{T_t} \right) \right\}.$$

Si la nueva configuración mejora el desempeño ($E(x') < E(x_t)$), se acepta siempre. Si empeora, puede aceptarse con una probabilidad dependiente de la temperatura T_t , permitiendo escapar de óptimos locales.

En este contexto, la temperatura controla la exploración del espacio de hiperparámetros. Para temperaturas altas, el algoritmo acepta configuraciones peores con frecuencia, explorando arquitecturas diversas. A medida que la temperatura disminuye, la búsqueda se concentra alrededor de configuraciones prometedoras, refinando la solución final.

El programa de enfriamiento utilizado es geométrico:

$$T_{t+1} = \gamma T_t, \quad \gamma \in (0, 1),$$

lo que proporciona un equilibrio entre exploración global y explotación local en problemas de ajuste automático de hiperparámetros.

5. Algoritmos Evolutivos

La búsqueda local estocástica (SLS) mantiene una única mejor estimación en cada paso, x_t . Si se ejecuta durante T pasos y se reinicia K veces, el costo total es TK . Una alternativa natural es mantener un conjunto o población de K candidatos, S_t , que se intenta mejorar en cada paso. Esto se denomina *algoritmo evolutivo* (EA). Si se ejecuta durante T pasos, también toma TK tiempo; sin embargo, puede obtener mejores resultados que SLS con múltiples reinicios, ya que el procedimiento explora más del espacio en paralelo y comparte información entre los miembros de la población.

Los EA se inspiran en la evolución biológica, y adoptan su terminología. La *aptitud* de un miembro de la población es el valor de la función objetivo. Los descendientes en el paso $t + 1$ pueden crearse mediante mutación aleatoria de un padre (reproducción asexual) o combinando dos padres (recombinación o cruce, seguido de mutación).

El procedimiento por el cual se eligen los padres se denomina *función de selección*. Algunas variantes comunes son:

- **Selección por truncamiento:** los padres se eligen de los K individuos más aptos (conjunto élite).
- **Selección por torneo:** cada padre es el más apto de un subconjunto aleatorio de K individuos.
- **Selección proporcional a la aptitud:** cada padre se elige con probabilidad proporcional a su aptitud relativa.
- **Evolución regularizada:** se eliminan los individuos más antiguos y se seleccionan padres en función de su aptitud.

Además de la regla de selección, se deben especificar las reglas de recombinación y mutación. Algunas variantes destacadas son:

- **Algoritmos genéticos (GA):** utilizan mutación y recombinación basada en cruce de cadenas binarias o vectores de enteros.

- **Programación genética:** los individuos se representan como árboles, lo que asegura que los cruces produzcan descendientes válidos.
- **EA asistidos por sustituto:** se emplea una función aproximada $\hat{f}(s)$ en lugar de la función objetivo real $f(s)$ para acelerar la evaluación.
- **Algoritmos meméticos:** combinan mutación y recombinación con búsqueda local estándar.

Los algoritmos evolutivos se han aplicado en numerosos campos, incluyendo el entrenamiento de redes neuronales, donde esta combinación se conoce como *neuroevolución*.

Algorithm 2 Algoritmo Genético Básico

```

1: Inicializar una población  $S_0$  de  $K$  individuos
2: for  $t = 1, \dots, T$  do
3:   Evaluar la aptitud de cada individuo  $x \in S_{t-1}$  mediante la función objetivo  $f(x)$ 
4:   Seleccionar padres de  $S_{t-1}$  según una regla de selección (ej. proporcional a la aptitud)
5:   Aplicar cruce:
6:   for cada par de padres seleccionados do
7:     Elegir un punto de cruce aleatorio
8:     Intercambiar segmentos de los cromosomas para generar descendientes
9:   end for
10:  Aplicar mutación:
11:  for cada descendiente do
12:    Con probabilidad  $p_{mut}$ , alterar aleatoriamente un gen
13:  end for
14:  Formar la nueva población  $S_t$  con los descendientes (y opcionalmente algunos individuos elite)
15: end for
16: Devolver el mejor individuo encontrado

```

5.1. Aplicación: optimización de hiperparámetros

En la optimización de hiperparámetros, cada individuo de la población representa una configuración completa de una red neuronal. Un cromosoma se define como

$$x = (\eta, \lambda, o, a, h),$$

donde η es la tasa de aprendizaje, λ el parámetro de regularización, o el optimizador, a la función de activación y h la arquitectura oculta.

La aptitud de cada individuo se obtiene a partir del desempeño del modelo entrenado con esa configuración. Si f_x representa la red asociada al individuo x , entonces

$$\text{fitness}(x) = -\mathcal{L}_{\text{valid}}(f_x),$$

de modo que minimizar la pérdida de validación equivale a maximizar la aptitud.

La selección de padres se realiza de forma probabilística. Sea f_i la aptitud del individuo x_i , entonces su probabilidad de selección se define mediante la transformación softmax:

$$p_i = \frac{\exp(f_i)}{\sum_{j=1}^N \exp(f_j)},$$

donde N es el tamaño de la población. A partir de estas probabilidades se seleccionan k padres, favoreciendo individuos con mejor desempeño pero conservando diversidad en la población.

El operador de cruce combina hiperparámetros de dos padres. Si $x^{(1)}$ y $x^{(2)}$ son dos configuraciones parentales, el descendiente se construye tomando aleatoriamente cada componente de uno de ellos:

$$x'_j = \begin{cases} x_j^{(1)}, & \text{con probabilidad } 1/2, \\ x_j^{(2)}, & \text{con probabilidad } 1/2. \end{cases}$$

La mutación introduce variación aleatoria. Para hiperparámetros continuos, como η y λ , se aplican perturbaciones gaussianas sobre sus transformaciones logarítmicas; para hiperparámetros discretos, como optimizador o activación, se cambia aleatoriamente la categoría; para la arquitectura, se modifica el número de capas o el número de neuronas de una capa seleccionada.

De esta forma, el algoritmo evolutivo explora simultáneamente múltiples configuraciones, combinando explotación mediante selección y cruce, y exploración mediante mutación.

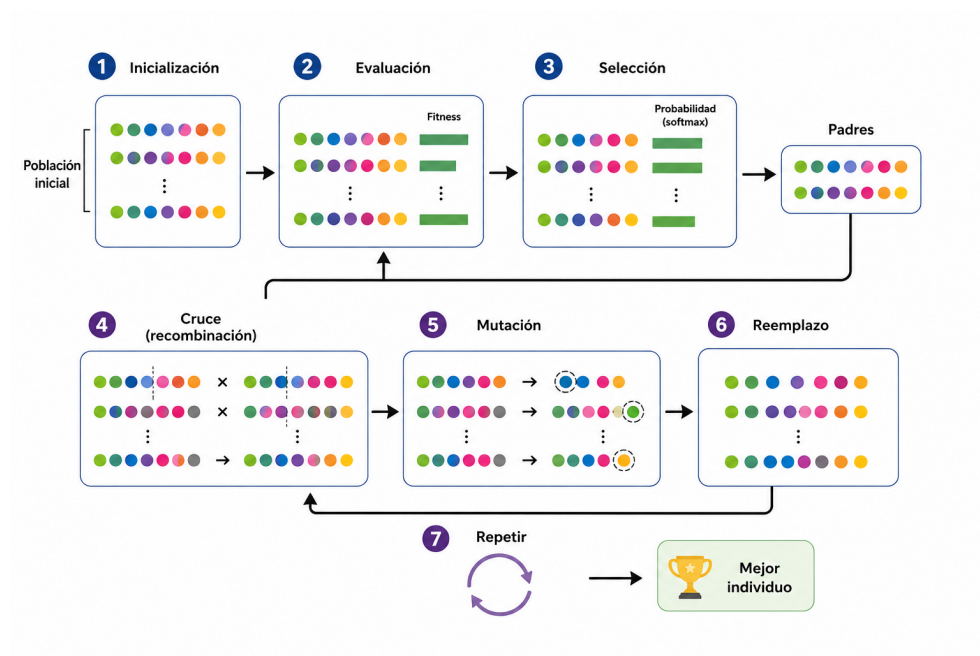


Figura 1